

Exercise : Code documentation

1. I selected the `calculateTaskScore(task)` function from the `task priority.js` file. I chose this algorithm because it calculates a task's priority score using several factors, including task priority, due date, status, tags, and the time since the task was last updated.

2. `calculateTaskScore(task)`

Description:

This function calculates how important a task is by giving it a score. It looks at the task's priority, due date, status, tags, and when it was last updated.

Parameter:

- `task` - The task that needs to be scored. It contains the task's priority, due date, status, tags, and update date.

Return value:

- Returns a number representing the task's importance score. A higher score means the task gets more priority.

Possible errors:

- An error may occur if the task or its information is missing or in the wrong format.

Example:

```
```javascript
const score = calculateTaskScore(task);
console.log(score);
```

### Important notes:

- Higher-priority tasks get more points.
- Overdue or soon-due tasks get extra points.
- Completed tasks lose points.
- Tasks with tags such as urgent, critical, or blocker get extra points.
- Recently updated tasks get extra points.
- A task without a due date does not get any due-date points.

3. I reviewed the generated documentation and compared it with the `calculateTaskScore()` code. The documentation correctly explains that the function calculates a task score using priority, due date, status, tags, and the last update time.

It also includes the parameter, return value, possible errors, example usage, and important notes. The information matches the code, so the documentation is accurate and complete.

4. I adjusted the prompt to ask for simple and clear documentation that is easy for a beginner to understand. I also asked the AI to focus only on the `calculateTaskScore()` function and to explain each part of the function in simple language.

5. I used Prompt 2 on the `calculateTaskScore()` function to get a different explanation of the code.

The prompt helped me understand that the function is trying to calculate how important a task is by combining several factors. It starts with the task's priority and then changes the score depending on the due date, task status, tags, and how recently the task was updated.

The step-by-step explanation also helped me identify edge cases, such as a task without a due date, an unknown priority, or invalid date information.

The prompt also suggested useful comments that could be added to the code to explain the more complicated parts of the scoring process.

6. I compared the results from Prompt 1 and Prompt 2. Prompt 1 gave a clear description of the function, its parameter, return value, possible errors, and example usage. Prompt 2 gave a better explanation of the step-by-step logic, assumptions, edge cases, and possible improvements.

I combined these useful parts to create a more complete explanation of `calculateTaskScore()`. The final documentation explains what the function does, how the score is calculated, possible edge cases, and how the code could be improved while keeping its original purpose.

## 7. `calculateTaskScore(task)`

### Purpose:

This function calculates an importance score for a task. It uses the task's priority, due date, status, tags, and last update time.

### How it works:

1. It gives the task a starting score based on its priority.
2. It adds points if the task is overdue or due soon.
3. It removes points if the task is completed or in review.
4. It adds points if the task has important tags such as urgent, critical, or blocker.

5. It adds points if the task was updated recently.
6. It returns the final score.

Parameter:

- task - The task that needs to be scored. It should contain priority, due date, status, tags, and update information.

Return value:

- Returns a number representing the task's importance score.

Possible edge cases:

- A task without a due date does not receive due-date points.
- An unknown priority gives a base score of 0.
- The tags property should be an array.
- Invalid date values may cause incorrect results.
- The score can change as time passes because the function uses the current date and time.

Example:

```
const score = calculateTaskScore(task);
console.log(score);
```

Possible improvements:

The code could validate the task information before calculating the score. The different scoring values could also be stored separately so they are easier to change in the future.

Discussion of differences :

The first documentation version focused more on describing the function, its parameters, return value, errors, and example usage.

The second version focused more on explaining the logic step-by-step, assumptions, edge cases, comments, and possible improvements.

The final version combines the useful parts of both. It is more complete than the first version and easier to understand than a highly technical explanation. I also kept the language simple so that the purpose and logic of the function are clear.